

Adversarial Sinks

Sterowanie zbieżnością ataków adversarialnych poprzez techniki obronne i wprowadzanie wzorców typu backdoor

Maksymilian Baj
Franciszek Zaranowicz

Dokumentacja końcowa — ZZSN, Lato 2026 (Projekt nr 8)

Spis treści

1. Wprowadzenie i motywacja	2
2. Tło i sformułowanie problemu	2
2.1. Model zagrożenia i ataki	2
2.2. Proponowana funkcja straty	2
2.3. Co oznacza „sukces”: metryki detekcji	3
3. Baza kodu	3
4. Przebieg eksperymentów	4
4.1. Faza 1: poprawienie mechanizmu i obserwacja, jak walczy sam ze sobą	4
4.2. Faza 2: osadzanie sinka jako triggera (CrossTrap i BadNets)	5
4.3. Faza 3: ograniczanie lokalizacji zamiast kształtu	5
4.4. Faza 4: wykluczenie czynników zakłócających za pomocą środowiska toy i przeglądu pojemności	5
4.5. Faza 5: „wygrana” w środowisku toy i precyzyjna granica	7
4.6. Faza 6: powrót na CIFAR, tym razem wiernie	8
5. Synteza i werdykt	12
6. Wnioski i kierunki przyszłych prac	13
Bibliografia	14

1. Wprowadzenie i motywacja

Przykłady adwersarialne to niewielkie, celowo spreparowane perturbacje, które zmieniają predykcję sieci, pozostając przy tym niemal niewidoczne dla człowieka [1]. Atakujący w scenariuszu white-box, mający dostęp do gradientu modelu, jest w stanie niezawodnie przepchnąć dowolne wejście przez granicę decyzyjną, podążając za kierunkiem, w którym strata klasyfikacji rośnie najszybciej [2].

Geneza projektu to proste, nieco przewrotne pytanie. Zamiast uodparniać model — co jest zwykłym celem treningu adwersarialnego [3] — czy możemy sprawić, by model *zdradził atakującego*? Konkretnie: czy da się ukształtować krajobraz strat tak, aby perturbacja znaleziona przez atak nie była typowym quasi-szumem, lecz ustalonym, zaprojektowanym z góry wzorcem wizualnym — *adversarial sink* (np. symbolem „X”)? Gdyby udany atak był *zmuszony* narysować znany symbol, każda próba manipulacji stałaby się natychmiast i wizualnie wykrywalna.

Urok tego pomysłu polega na obróceniu siły atakującego przeciwko niemu: to właśnie podążanie za gradientem, które czyni ataki white-box skutecznymi, kierowałoby je prosto w pułapkę. Nasz plan łączył trzy obronne składowe w jedną niestandardową funkcję straty, ocenianą na zbiorze CIFAR-10.

Niniejszy raport jest celowo chronologiczny. Główny wynik jest częściowo negatywny, a prawdziwym wkładem jest *tok rozumowania, który do niego doprowadził* — sekwencja wypróbowanych mechanizmów, przyczyny porażki każdego z nich oraz płynące z nich wnioski. Kończymy precyzyjną charakterystyką tego, co jest, a co nie jest osiągalne, oraz konkretnymi kierunkami dalszych prac.

2. Tło i sformułowanie problemu

2.1. Model zagrożenia i ataki

Pracujemy w standardowym scenariuszu white-box. Klasyfikator f_θ odwzorowuje obraz $x \in [0, 1]^D$ (tutaj $D = 3 \cdot 32 \cdot 32 = 3072$) na logity klas. Atak poszukuje perturbacji δ , ograniczonej budżetem $\|\delta\| \leq \varepsilon$, która maksymalizuje stratę klasyfikacji. Używamy dwóch kanonicznych ataków pierwszego rzędu, oba przez bibliotekę Foolbox [4]: FGSM — pojedynczego kroku w kierunku znaku gradientu [2] — oraz PGD, jego iterowanej i rzutowanej wersji [3].

Iteracja PGD (w wariancie L_∞) ma postać:

$$\delta_{t+1} = \text{clip}_\varepsilon(\delta_t + \alpha \cdot \text{sign}(\nabla_{\delta} \mathcal{L}_{\text{CE}}(f_\theta(x+\delta_t), y))) \quad (1)$$

Istotna decyzja podjęta na wczesnym etapie: rzadki, wysokokontrastowy sink („X” na kilku pikselach) może zostać odtworzony wyłącznie przez atak L_2 . Atak L_∞ wysyca *każdy* piksel do $\pm\varepsilon$, więc nigdy nie namaluje rzadkiego kształtu. Dodaliśmy zatem wariant PGD w normie L_2 (rzutowanie każdej próbki na kulę L_2) i używamy go we wszystkich ewaluacjach sinka.

2.2. Proponowana funkcja straty

Mechanizm jest zgodny ze specyfikacją projektu: niestandardowa funkcja celu nakładająca trzy składniki na zwykłą klasyfikację entropią krzyżową.

Dopasowanie gradientu (gradient alignment). Aby nagiąć gradient względem wejścia w stronę sinka X , karcemy kąt między nimi. Przy podobieństwie kosinusowym składnik wynosi 0, gdy gradient już wskazuje na sink, i rośnie ku 2, gdy wskazuje przeciwnie:

$$L_{\text{align}} = 1 - \frac{\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y) \cdot X}{\|\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y)\|_2 \cdot \|X\|_2} \quad (2)$$

Utrzymanie sinka (sink preservation). Chcemy, aby model wciąż błędnie klasyfikował, gdy sink jest nałożony na obraz, tak by pozostawał on niezawodną „dziurą” w zabezpieczeniach. To składnik ujemny, utrzymujący $\mathcal{L}_{\text{CE}}(f_{\theta}(x + X), y)$ na wysokim poziomie:

$$L_{\text{sink}} = -\mathcal{L}_{\text{CE}}(f_{\theta}(x + X), y) \quad (3)$$

Trening ortogonalny (orthogonal adversarial training). Klasyczny trening adwersarialny w stylu Mądrego [3] uodparnia model we *wszystkich* kierunkach, w tym w kierunku sinka, co jest sprzeczne ze składnikiem (2). Zamiast tego rzutujemy perturbację PGD na podprzestrzeń ortogonalną do X , więc odporność trenowana jest w każdym kierunku *poza* sinkiem:

$$L_{\text{robust}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y), \quad \delta^{\perp} = \delta_{\text{PGD}} - \text{proj}_X(\delta_{\text{PGD}}) \quad (4)$$

Pełna funkcja celu łączy te składniki za pomocą trzech hiperparametrów równoważących $\alpha, \lambda_s, \lambda_r$:

$$L_{\text{total}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x), y) + \alpha L_{\text{align}} - \lambda_s \mathcal{L}_{\text{CE}}(f_{\theta}(x + X), y) + \lambda_r \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y) \quad (5)$$

2.3. Co oznacza „sukces”: metryki detekcji

„Wykrywalny” to nie to samo co „widoczny dla człowieka”. Nie potrzebujemy, by atak wyrysował wyraźny „X”; potrzebujemy, by jego perturbacja δ trafiała w znany wzorec w mierzalnym stopniu, znacznie powyżej poziomu losowego. Dla sinka s o nośniku S (jego niezerowych pikselach) śledzimy trzy uzupełniające się wielkości:

- **support_cos** — kosinus między δ a *znakowanym* szablonem: czy δ odtwarza dokładny kształt i znak? Poziom losowy ≈ 0 .
- **mass_frac** — udział masy bezwzględnej (L_1) perturbacji δ przypadający na nośnik S . Poziom losowy $= |S| / D$, czyli masa, jaką nakłada tam jednorodnie losowe δ .
- **energy_frac** — udział energii kwadratowej (L_2) perturbacji δ wzdłuż kierunku sinka, czyli $\cos^2(\delta, s)$. Poziom losowy $= 1/D$ dla kierunku jednowymiarowego. To metryka rozstrzygająca dla *gęstych* sinków, gdzie dwie pozostałe są nieinformacyjne.

Wzorec uznajemy za *narysowany* wtedy i tylko wtedy, gdy **support_cos** jest wyraźnie dodatni i **mass_frac** przekracza poziom losowy, przy dokładności na czystych przykładach wartej zachowania. *Koncentracja energii* to słabsze stwierdzenie klasy detekcyjnej: **energy_frac** $\gg$ poziom losowy, nawet jeśli dokładny znak i kształt nie zostają odtworzone.

3. Baza kodu

Ponieważ jest to raport projektowy, a nie wyłącznie analiza wyników, krótko opisujemy zbudowane przez nas oprogramowanie. Całość jest wielokrotnego użytku i była intensywnie wykorzystywana w blisko dwudziestu eksperymentach.

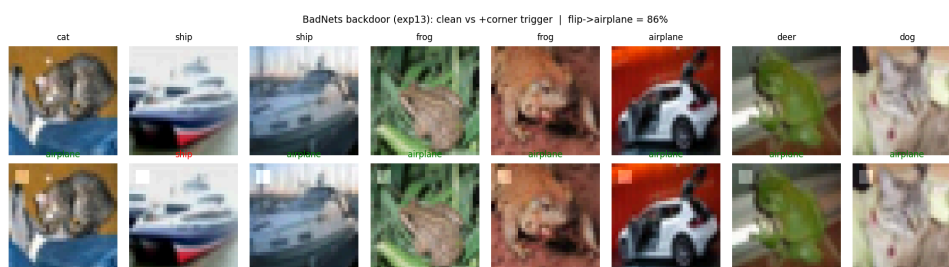
Rdzeniem jest importowalny pakiet Pythona `adversarial_sinks`, oparty na PyTorch i PyTorch Lightning. Jego sercem jest pojedynczy *potok* (pipeline), który przyjmuje wzorec sinka i funkcję straty oraz przeprowadza eksperyment od początku do końca — trening → atak → metryki → raport. Wokół niego:

- **losses.py** — każdy wypróbowany mechanizm jako wymienna klasa straty (`AdversarialSinkLoss`, `CrossTrapLoss`, `BadNetPoisonLoss`, `SinkConfinementLoss`); potok jest niezależny od wyboru straty.
- **attacks.py** — FGSM oraz PGD (L_2 i L_{∞}) przez Foolbox, z rzutowaniem dokładnie respektującym budżet.
- **sink_patterns.py** — biblioteka wzorców (pełny krzyż, `small_cross`, `corner_square`, `constellation`, szachownice) oraz gęste kierunki „void”.
- **metrics.py** — trzy metryki detekcji i statystyki per-próbką.

- sieć CNN z pokrętle szerokości (`base_channels`), pozwalającym skalować pojemność, z normalizacją wejścia przeniesioną do wnętrza sieci, tak by każda strata i każdy atak działały w tej samej przestrzeni pikseli $[0, 1]$.

Dwa praktyczne ograniczenia ukształtowały całość. Po pierwsze, cały trening odbywał się wyłącznie na CPU (bez CUDA), a składnik dopasowania wymaga gradientu drugiego rzędu, co kosztuje ~ 2.9 s na partię; eksperymenty są więc wymiarowane w setkach partii i wznawialne dzięki znacznikom zapisywanym na dysku. Po drugie, osobne, w pełni zbieżne dwuwymiarowe środowisko *toy* pozwala narysować cały krajobraz strat, pole gradientu oraz trajektorie ataku na żywo — było to narzędzie o najwyższej ilości informacji na sekundę i to właśnie ono ostatecznie oddzieliło przyczynę od czynnika zakłócającego.

Wszystkie prace nad sinkiem poprzedziła kontrola poprawności: sprawdziliśmy, że ataki zachowują się prawidłowo (PGD jest monotoniczne względem ϵ , dokładnie respektuje budżet i jest znacznie silniejsze niż losowy szum; model jest odporny na losowy szum) oraz że podręcznikowy backdoor BadNets [5] trenuje się poprawnie w naszym środowisku (Rysunek 1).



Rysunek 1: **Kontrola poprawności backdoora BadNets.** Model wytrenowany ze standardowym triggerem zatrującym dane BadNets uczy się skrótu „obecny trigger $\rightarrow$ klasa docelowa”. Nałożenie triggera w rogu podnosi przewidywane prawdopodobieństwo klasy docelowej *samolot* do 0.855, wobec 0.137 na czystym obrazie, przy zachowanej dokładności na czystych przykładach. Potwierdza to, że nasze środowisko potrafi osadzić backdoor; pytaniem, na które odpowiada reszta raportu, jest to, czy *atak* gradientowy da się zmusić do samodzielnego odtworzenia takiego triggera.

4. Przebieg eksperymentów

Przedstawiamy pracę w kolejności, w jakiej miała miejsce, ponieważ każdy zwrot był wymuszony przez poprzednią porażkę. Wyłaniają się dwie rodziny podejść: *sterowanie atakiem ku ustalonemu celowi*, którą wyczerpujemy, oraz *koncentracja energii ataku w znanej podprzestrzeni*, która częściowo działa. Granica między nimi jest głównym wynikiem.

4.1. Faza 1: poprawienie mechanizmu i obserwacja, jak walczy sam ze sobą

Pierwsze tygodnie poświęciliśmy temu, by proponowana strata rzeczywiście robiła to, co deklaruje. Cztery strukturalne błędy po cichu ją neutralizowały: niezgodność przestrzeni normalizacji między zbiorem danych a składnikami $\text{clip}(0, 1)$; brak flagi drugiego rzędu, przez który składnik dopasowania był pustą operacją wnoszącą zerowy gradient do wag; błąd znaku w składniku sinka, który trenował odporność *na* sink zamiast podatności; oraz nieograniczony składnik ujemny, który rozbiega się do nieskończoności, a który okiełznaliśmy ograniczeniem (*margin clamp*).

Gdy mechanizm wreszcie działał, natychmiast pojawiło się centralne napięcie — i nie był to błąd. Przy $\alpha = 1.0$ składnik dopasowania ledwie drgnie: gradient klasyfikacji dominuje, a strata dopasowania pozostaje bliska 0.99. Większe α faktycznie zmniejsza kąt, ale tylko kosztem degradacji klasyfikacji — gradient sieci względem wejścia nie może jednocześnie kodować klasy *i* wskazywać ustalonego, niezależnego od wejścia kierunku. Ten konflikt — *gradient-koduje-klasę* kontra *gradient-wskazuje-sink* — przewija się przez cały projekt.

4.2. Faza 2: osadzanie sinka jako triggera (CrossTrap i BadNets)

Skoro nie umiemy nagiąć gradientu, być może da się osadzić sink jako trigger typu backdoor, tak by atak go *odkrył*. CrossTrapLoss traktuje krzyż jako ukierunkowaną perturbację uniwersalną — krzyż + dowolny obraz → ustalona klasa — z treningiem ortogonalnym uodparniającym każdy inny kierunek. Doszło do załamania do $\sim 10\%$ dokładności na czystych przykładach. Diagnostyczny przegląd bez ataku pokazał, że to problem *wagi*, a nie strojenia: składnik pułapki jest trywialnie spełnialny, podczas gdy klasyfikacja jest trudna, więc optymalizator całkowicie porzuca klasyfikację.

BadNetPoisonLoss usunął załamanie, zatruwając jedynie niewielki ułamek każdej partii i składając go w pojedynczą entropię krzyżową, tak że czysta większość zachowuje dokładność. Trening przebiegł poprawnie (0.64 dokładności na czystych przykładach) — ale PGD nie narysował triggera: `mass_frac` utrzymywał się na poziomie losowym lub *poniżej* niego przy każdym budżecie. Atak aktywnie unika triggera.

Ta porażka wymusiła najważniejszy mechanistyczny wniosek całego projektu.

Kluczowy wniosek. PGD podąża za *lokalnym* gradientem wejścia $\partial\mathcal{L}/\partial x$ w punkcie czystym. Backdoor to *skończona, nieliniowa* reakcja „jeśli trigger obecny → przełącz”; nie tworzy on lokalnego gradientu wskazującego na trigger. Dlatego atak wspinający się lokalnie nigdy tam nie dochodzi — podąża za resztkowymi gradientami na istotnych pikselach *obiektu*, co dokładnie tłumaczy, czemu energia koncentruje się *z dala* od narożnika czy tła o niskim gradiencie, gdzie żyją triggerzy. Trening ortogonalny spłaszcza inne kierunki, lecz nie *tworzy* gradientu ku sinkowi; a jedyny mechanizm, który by go stworzył — dopasowanie — walczy z klasyfikacją i przegrywa. Sterowanie przestrzenne i sterowanie kierunkowe to dwa spojrzenia na tę samą ścianę.

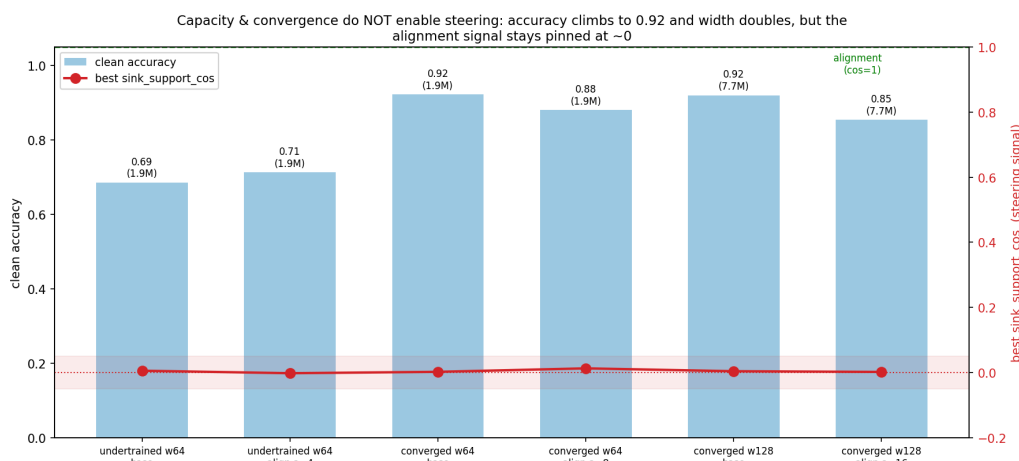
4.3. Faza 3: ograniczanie lokalizacji zamiast kształtu

Cel łagodniejszy: przestać żądać od PGD rysowania znakowanego szablonu i zamiast tego ograniczyć jego energię do znanego *obszaru*. SinkConfinementLoss uogólnia trening ortogonalny z wyjątku jednowymiarowego (szablon) do wyjątku obejmującego całą podprzestrzeń przestrzenną — maskując perturbację PGD wewnątrz obszaru przed składnikiem odporności — plus backdoor wewnątrz, by obszar pozostał atakowalny. To również zawiodło: `mass_frac` poniżej poziomu losowego, `support_cos` ≈ 0 . Maskowany trening uodparnia *poza* obszarem, ale nie tworzy przyciągania *do* niego, więc PGD nadal unika narożnika. Każdy mechanizm „sterowania ku ustalonemu celowi przestrzennemu lub szablonowi” — dopasowanie, CrossTrap, BadNets, maskowane ograniczanie — jest teraz wyczerpany.

4.4. Faza 4: wykluczenie czynników zakłócających za pomocą środowiska toy i przeglądu pojemności

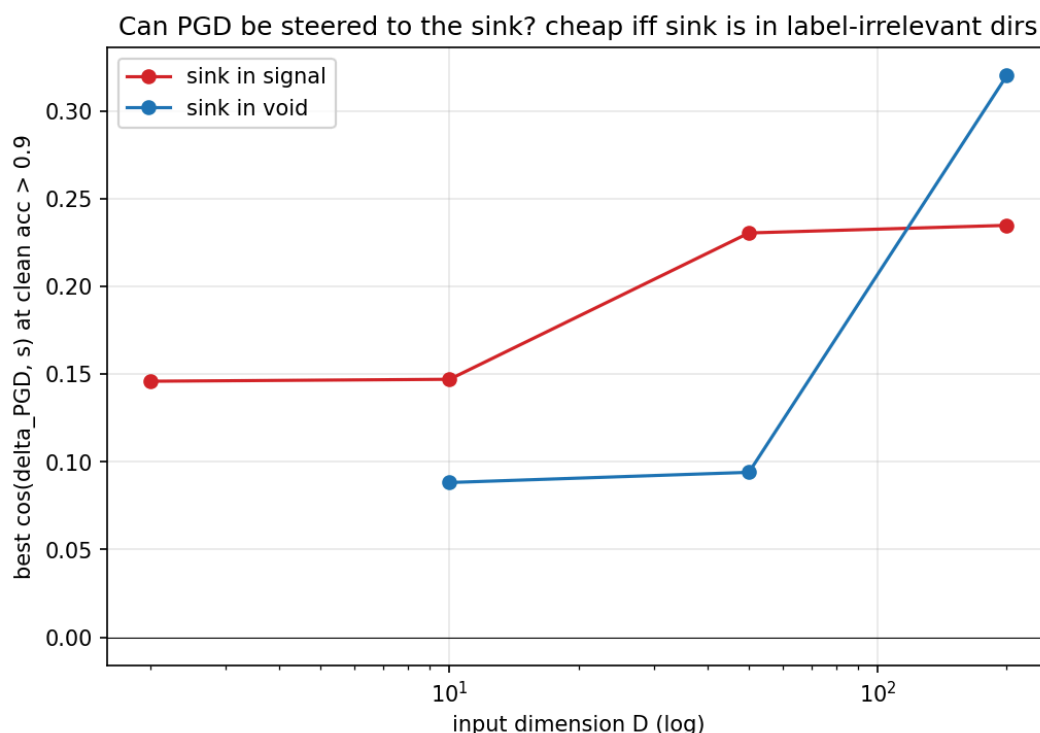
Zanim ogłosiliśmy pomysł niewykonalnym, musieliśmy wykluczyć dwa czynniki zakłócające: być może modele CIFAR były po prostu niedotrenowane na CPU (50–70% dokładności), albo sieci brakowało pojemności. Oba okazały się fałszywe, a właśnie ich wykluczenie czyni wynik negatywny wiarygodnym.

Przegląd pojemności rozstrzygnął sprawę. Użyta sieć CNN była już ResNetem o ~ 1.9 mln parametrów, więc pojemność nigdy nie była ograniczeniem — czynnikiem zakłócającym było niedotrenowanie. Wytrenowana do zbieżności, ta sama baza o szerokości 64 osiąga 0.923 dokładności, a $2 \times$ szersza baza o szerokości 128 osiąga 0.921. Dostrajanie samym dopasowaniem (isolated alignment) z każdej zbieżnej bazy nadal daje `support_cos` ≈ 0 (0.002–0.013) i `energy_frac` na poziomie losowym. Zbieżność i $4 \times$ większa pojemność *nie* odblokowują sterowania kierunkowego — co potwierdza napięcie strukturalne, a nie niedotrenowanie.



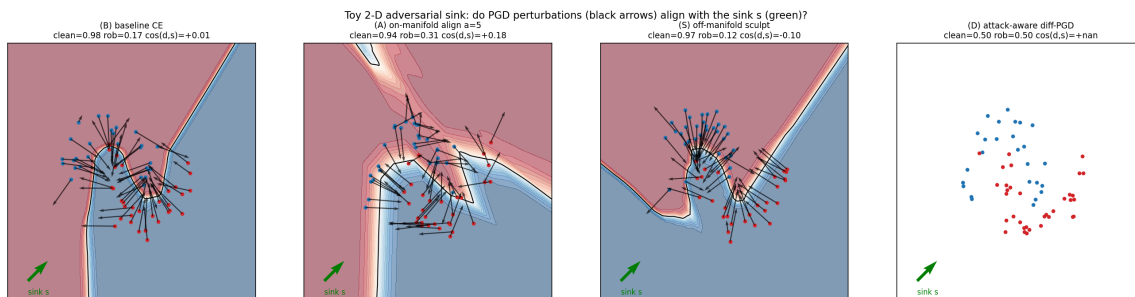
Rysunek 2: **Pojemność i zbieżność nie są ograniczeniem.** Dokładność na czystych przykładach dla różnych konfiguracji modelu: wczesne, niedotrenowane przebiegi o szerokości 64 (~ 0.69) leżą znacznie poniżej tej samej architektury wytrenowanej do zbieżności (0.923), a podwojenie szerokości do 7.7 mln parametrów nie pomaga (0.921). Co istotne, dostrajanie dopasowaniem na tych w pełni zbieżnych, pojemnych bazach *nadal* nie steruje atakiem ($\text{support_cos} \approx 0$), więc niemożność narysowania sinka jest strukturalna — nie tłumaczą jej słabe modele ani zbyt mało epok.

Środowisko toy zlokalizowało następnie przeszkodę dokładnie. W zbieżnym, dwuwymiarowym, dwuklasowym MLP wszystko jest widoczne. Wyróżniają się dwa wyniki. Po pierwsze, hipoteza wymiarowości zostaje obalona: najlepsze osiągalne dopasowanie *nie* pogarsza się wraz ze wzrostem wymiaru wejścia od 2 do 1000 — jest płaskie lub rosnące (Rysunek 3). Stwierdzenie „CIFAR zawodzi przez swoje 3072 wymiary” jest błędne.



Rysunek 3: **Jakość dopasowania nie maleje z wymiarem.** Najlepszy osiągalny $\cos(\delta, s)$ (oraz energia na osi sinka) w funkcji wymiaru wejścia D , dla sinka umieszczonego w podprzestrzeni istotnej dla etykiety („signal”) i nieistotnej („void”), w w pełni zbieżnych sieciach toy. Krzywe są płaskie lub rosnące, a nie malejące: to nie wysoka wymiarowość blokuje sterowanie. Umieszczenie sinka w podprzestrzeni „void” jest konsekwentnie łatwiejsze — to załóżek wyniku, który ostatecznie przenosi się na CIFAR.

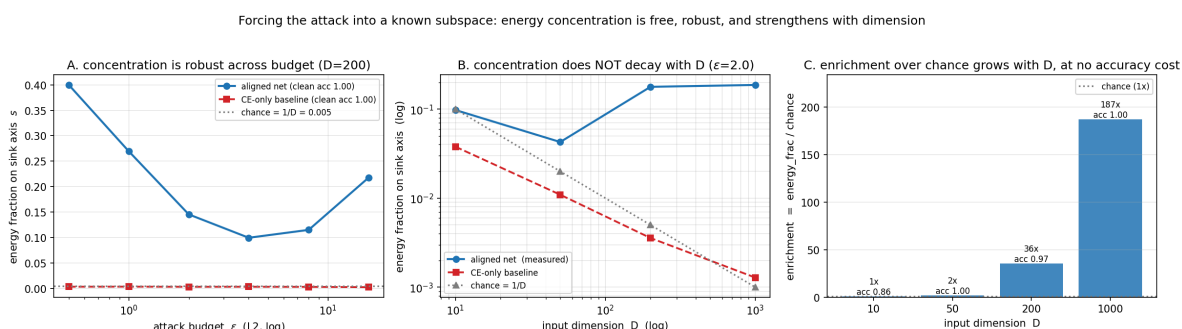
Po drugie, przegląd budżetu jest rozstrzygający. Energia ataku koncentruje się na wybranej osi jednowymiarowej znacznie powyżej poziomu losowego (20–33% wobec 0.5% dla $D = 200$, czyli $40\text{--}60 \times$ wzbogacenie), odpornie na zmianę budżetu ataku – a mimo to znakowane dopasowanie $\cos(\delta, s)$ nigdy nie przekracza ~ 0.27 , nigdy nie dominuje i przy dużym budżecie zmienia znak na *ujemny* (atak idzie przeciw sinkowi). Koncentracja energii jest darmowa; zmuszenie ataku do *narysowania* znakowanego, dominującego sinka jest fundamentalnie zablokowane.



Rysunek 4: **Krajobrazy strat i trajektorie ataku w środowisku toy.** Każdy panel pokazuje dwuwymiarową powierzchnię strat, pole gradientu wejścia, granicę decyzyjną oraz trajektorie PGD na żywo dla innego mechanizmu (baza, dopasowanie na różnorodności, rzeźbienie poza różnorodnością, wariant świadomy ataku). Dopasowanie na różnorodności odtwarza napięcie dokładność–sterowanie w w pełni zbieżnej sieci; warianty poza różnorodnością i świadome ataku nie biją zwykłego dopasowania i są niestabilne. Rysunek uwidacznia, dlaczego żadna trajektoria nie zostaje nagięta w dominującą studnię przy sinku.

4.5. Faza 5: „wygrana” w środowisku toy i precyzyjna granica

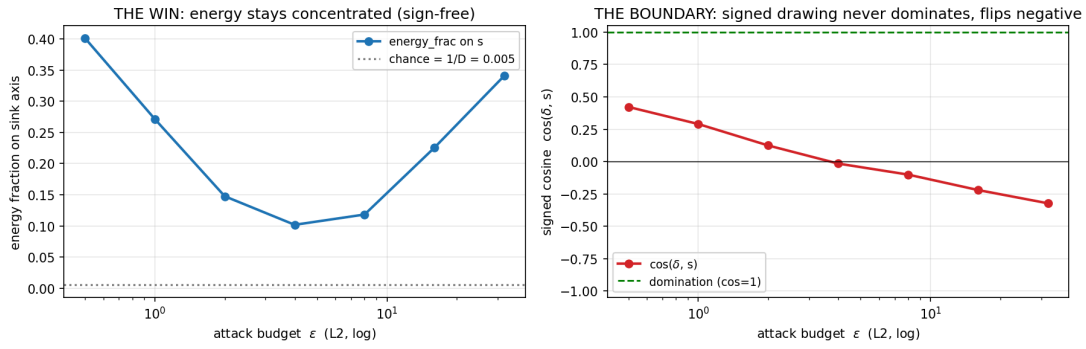
Przeformułowanie wyniku przeglądu budżetu na stwierdzenie pozytywne daje czysty wynik projektu. Jeśli porzucimy żądanie *rozpoznawalnego, znakowanego* sinka i poprosimy jedynie o to, by energia ataku trafiła na znany kierunek, środowisko toy dostarcza go spektakularnie i za darmo.



Rysunek 5: **Zmuszenie ataku do wejścia w znaną podprzestrzeń jest darmowe, odporne i rośnie z wymiarem.** (A) Udział energii ataku na wybranej osi sinka (1-D) w funkcji budżetu ataku ϵ ($D = 200$): sieć dopasowana (dokładność 1.00) utrzymuje $\sim 0.1\text{--}0.4$ całej energii ataku na pojedynczej osi w całym zakresie budżetu, znacznie powyżej linii losowej $1/D = 0.005$ i wyraźnie powyżej bazy uczonej samym CE. (B) Ten sam udział energii *nie* maleje, gdy wymiar wejścia D rośnie od 10 do 1000. (C) Wzbogacenie względem poziomu losowego *rośnie* więc z wymiarem – $1 \times$ przy $D = 10$, $2 \times$ przy 50, $36 \times$ przy 200, $187 \times$ przy 1000 – wszystko przy dokładności 0.86–1.00. Koncentracja energii to realny, odporny, darmowy efekt, gdy istnieją naprawdę nieużywane wymiary.

Ta sama zbieżna sieć równie wyraźnie pokazuje ścianę. Energia się koncentruje, lecz znakowany kosinus zmienia się od $+0.42$ do -0.32 wraz ze wzrostem budżetu: atak nigdy nie zobowiązuje się do znaku sinka i ostatecznie się z nim anty-dopasowuje. To właśnie granica, którą dostarczamy, potwierdzona w zbieżnej, niskowymiarowej sieci – rdzeń napięcia, a nie artefakt.

One aligned net (D=200, clean acc 1.00): concentration is free, signed domination is blocked



Rysunek 6: **Granica: energia się koncentruje, ale znak jest niekontrolowany.** Dla tej samej zbieżnej sieci toy, która daje Rysunek 5, energia na podprzestrzeni sinka pozostaje wysoka, lecz znakowane dopasowanie $\cos(\delta, s)$ spada od $+0.42$ do -0.32 wraz ze wzrostem budżetu ataku – perturbacja coraz bardziej wskazuje *przeciw* zamierzonemu sinkowi. Osiągalnym efektem jest więc bezznakowa koncentracja energii (detektor wciąż może ją zasygnalizować), podczas gdy czysty, dominujący, poprawnie znakowany rysunek jest fundamentalnie poza zasięgiem.

4.6. Faza 6: powrót na CIFAR, tym razem wiernie

Środowisko toy mówi, że koncentracja energii powinna być osiągalna; czy przenosi się na CIFAR? Sprawdziliśmy to w dwóch krokach na zbieżnej sieci o dokładności 0.92.

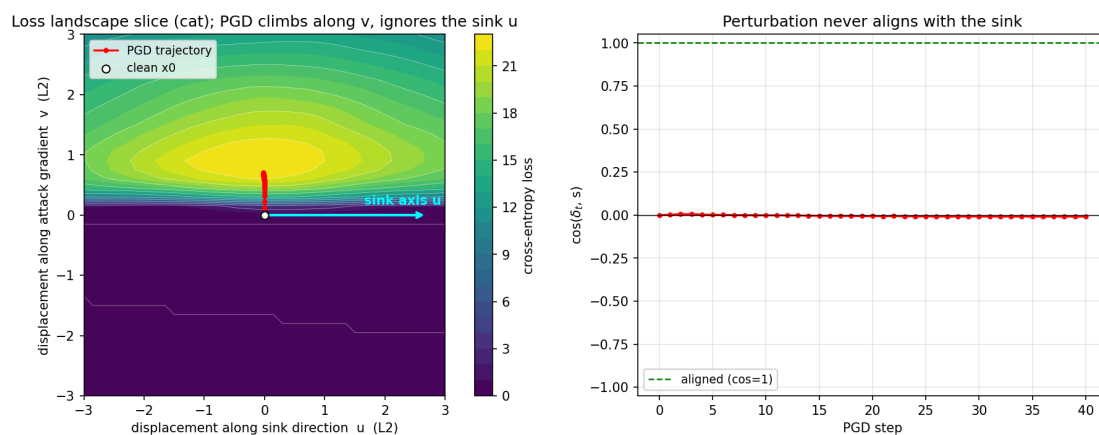
Po pierwsze, kontrolowany przegląd wzorców (pytanie Q5 z etapu 3). Wśród sześciu wzorców wizualnych (pełny krzyż, mały krzyż, konstelacja, kwadrat w rogu, dwie szachownice), przy najlepszym dostrajaniu dopasowaniem, *żaden* wzorec nie koncentruje energii na CIFAR: support_cos w przedziale $[-0.012, +0.013]$, mass_frac i energy_frac wszędzie na poziomie losowym – niezależnie od tego, czy centralny, peryferyjny, rzadki czy znakowany. Idea sinka wizualnego nie przenosi się.

wzorec	nośnik	losowo	dokł.	najl. mass_frac	werdykt
krzyż (pełny)	720	0.234	0.611	0.279	nie narysowany
krzyż (pełny), align FT	720	0.234	0.713	0.289	nie narysowany
small_cross 8×8	84	0.027	0.107	0.035	załamanie
corner_square 4×4, BadNet	48	0.016	0.642	0.019	nie narysowany
corner_square 4×4, +L2 AT	48	0.016	0.466	0.011	nie narysowany
corner_square 4×4, maska AT	48	0.016	0.532	0.011	nie narysowany

Tabela 1: **Złożoność wzorca a sterowalność na CIFAR-10.** Dla każdego wzorca i mechanizmu najlepszy mass_frac, jaki atak nakłada na nośnik wzorca, pozostaje na poziomie losowym $|S|/D$ lub poniżej, a support_cos (niepokazany) nigdy nie przekracza zera. Żadne rozmieszczenie – gęste czy rzadkie, centralne czy w rogu – nie zostaje narysowane; przypadki corner_square spadają nawet *poniżej* poziomu losowego, gdyż atak unika narożnika.

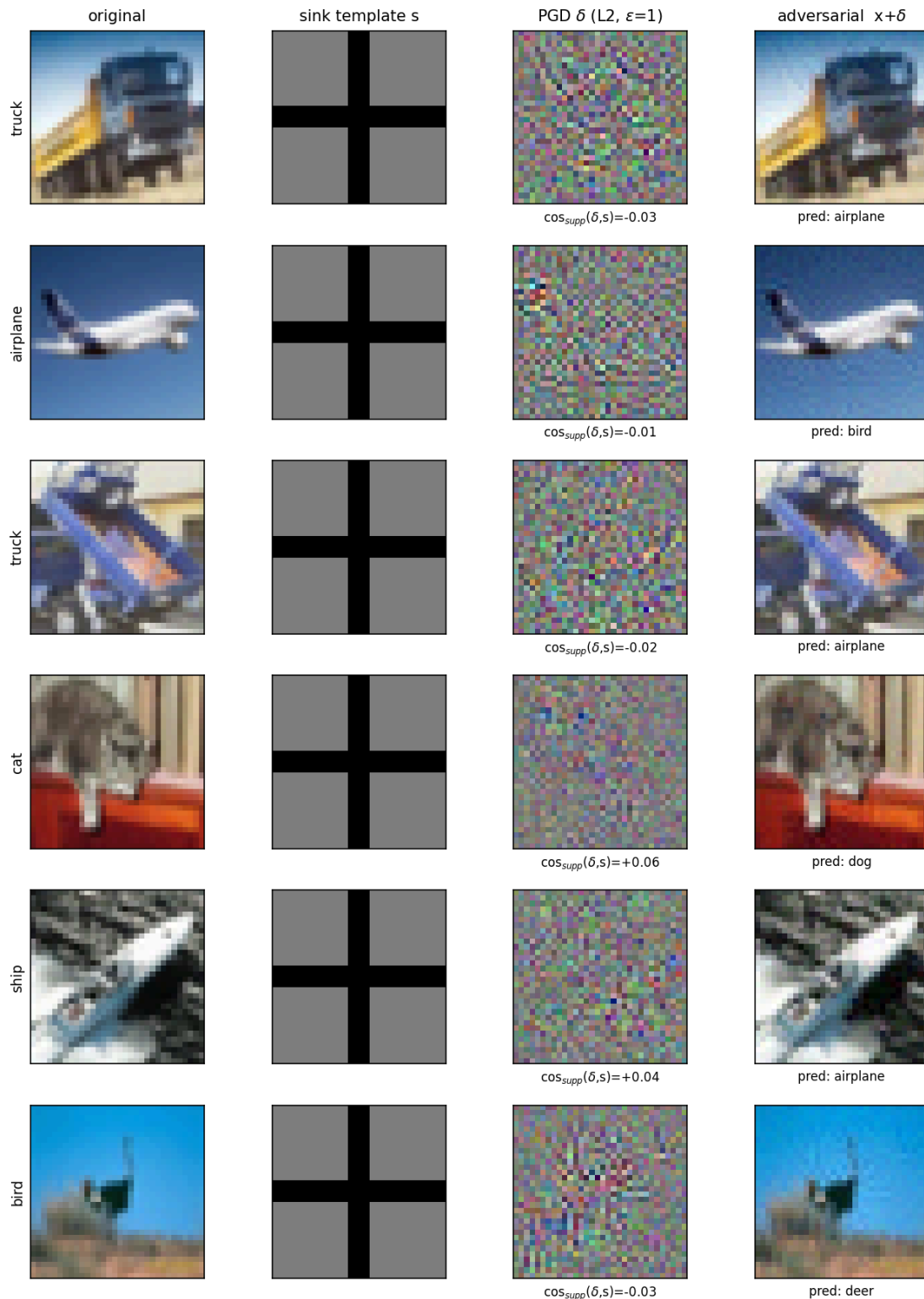
Potwierdziliśmy geometrię bezpośrednio. Przekrój krajobrazu strat w płaszczyźnie (sink, gradient) nie wykazuje studni w stronę sinka, a kosinus wzdłuż trajektorii PGD pozostaje płaski (Rysunek 7); zestawienie szablonu z rzeczywistą perturbacją pokazuje, że atak rysuje szum o kształcie obiektu (Rysunek 8).

No well toward the sink: the attack follows the object-gradient, not the planted direction



Rysunek 7: **Krajobraz strat CIFAR nie ma niecki w stronę sinka.** Dwuwymiarowy przekrój straty klasyfikacji rozpięty przez kierunek sinka i kierunek gradientu wejścia, z nałożoną trajektorią PGD oraz $\cos(\delta_t, s)$ w funkcji kroku ataku. Strata rośnie wzdłuż osi gradientu, lecz jest płaska wzdłuż osi sinka, trajektoria nigdy nie skręca ku sinkowi, a kosinus pozostaje ≈ 0 przez cały czas. Po prostu nie ma ścieżki w dół, którą atak mógłby podążać do sinka.

What the attack draws INSTEAD of the sink: PGD δ is diffuse object-noise, not the cross



Rysunek 8: **Co atak rysuje zamiast sinka.** Kolumny pokazują, dla kilku wejść, czysty obraz, zamierzony szablon sinka oraz rzeczywistą perturbację PGD. Perturbacja jest zorganizowana wokół istotnych pikseli *obiektu* – szum krawędzi i tekstur – i nie przypomina szablonu ($\cos \approx 0$). Atak wydaje budżet tam, gdzie lokalny gradient jest największy, czyli na obiekcie, nigdy na wzorcu projektanta.

FGSM zachowuje się tu tak jak PGD: oba dają $\text{support_cos} \approx 0$ i masę na poziomie losowym, choć PGD sprowadza dokładność odpornościową do zera (Tabela 2). Jedyne pozorne wyjątek — FGSM L_2 nakładający mass_frac 0.356 $>$ 0.234 na krzyż — to istotność pikseli centralnych, a nie rysowanie, gdyż support_cos pozostaje ~ 0 .

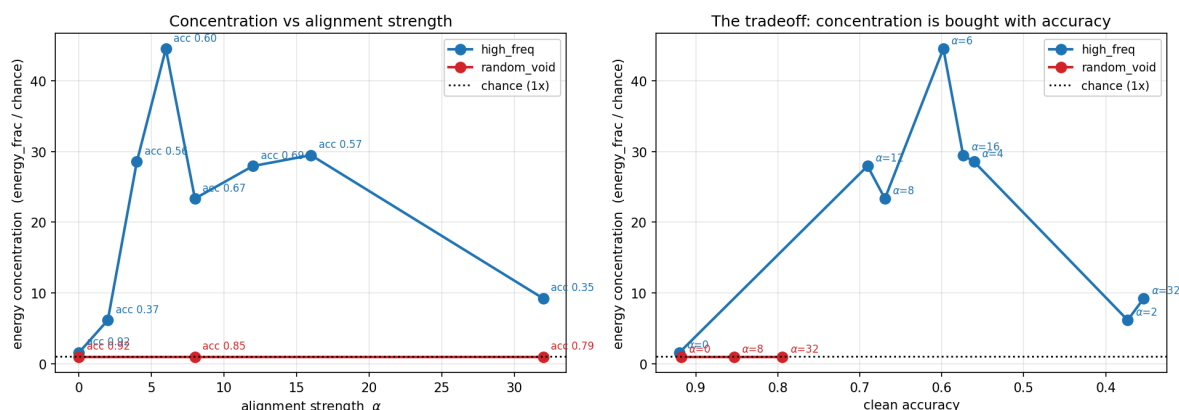
norma	atak	ϵ	dokł. odp.	support_cos	mass_frac
L2	FGSM	0.5	0.236	+0.000	0.356
L2	FGSM	2.0	0.146	−0.002	0.355
L2	PGD	0.5	0.012	+0.002	0.282
L2	PGD	2.0	0.000	+0.004	0.258
L_∞	FGSM	0.031	0.068	+0.006	0.236
L_∞	PGD	0.031	0.000	+0.004	0.238

Tabela 2: **FGSM i PGD są zgodne: brak rysowania, na zbieżnej sieci 0.92.** Dla obu ataków i obu norm $\text{support_cos} \approx 0$, a mass_frac jest na poziomie losowym (0.234 dla krzyża). PGD jest znacznie silniejszy jako atak (dokładność odpornościowa $\rightarrow 0$), lecz nie bardziej sterowalny. Podwyższona masa FGSM L_2 odzwierciedla istotność pikseli centralnych pokrywających się z krzyżem, a nie odtworzenie jego kształtu.

Po drugie, wierne przeniesienie idei toy: umieszczenie sinka w kierunku *nieistotnym dla etykiety*, na który klasyfikator jest naprawdę ślepy. Dopasowaliśmy gradient zbieżnej sieci ku gęstemu kierunkowi wysokoczęstotliwościowemu — szachownicy Nyquista piksel po pikselu, gdzie obrazy naturalne niosą niemal zero energii — oraz, dla kontroli, ku kierunkowi losowemu, przemiatając α .

To tutaj koncentracja wreszcie przenosi się na CIFAR — ale tylko dla kierunku wysokoczęstotliwościowego i nie za darmo. Względem poziomu losowego ($\text{energy_frac} = 3.26 \times 10^{-4}$) baza bez dopasowania niesie już $1.5 \times$ (pasmo wysokich częstotliwości naturalnie zawiera nieco więcej energii adversarialnej), a dopasowanie podnosi to do szczytu $\sim 44 \times$ przy $\alpha = 6$, z $\sim 23\text{--}28 \times$ utrzymanym przy $\alpha = 8\text{--}12$. Kierunek losowy pozostaje płaski na poziomie losowym dla każdego α : koncentracja wymaga kierunku, na który klasyfikator jest *ślepy*, a nie jedynie „niewizualnego”.

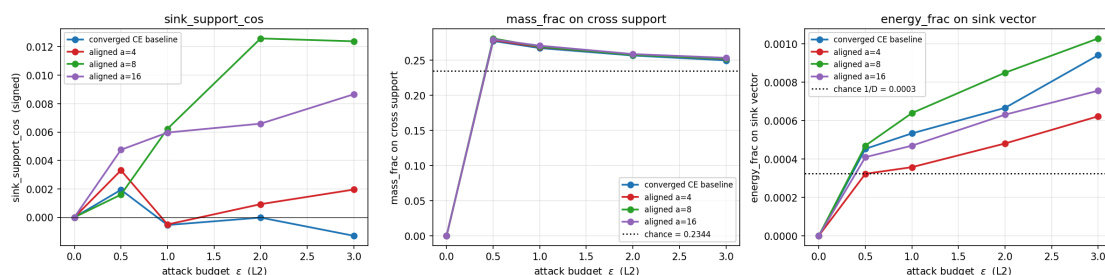
Koszt jest dokładność, a granica jest uderzająco niemonotoniczna. Wraz ze wzrostem α model najpierw się *załamuje* ($\alpha = 2 \rightarrow 0.38$ dokładności, powtarzalnie dla trzech ziaren), następnie *wraca do formy* przez $\alpha = 4\text{--}12$ (szczyt 0.69), po czym znów się *załamuje* ($\alpha = 32 \rightarrow 0.35$). Nie ma taniego kolana o wysokiej dokładności: każde dopasowanie na tyle silne, by skoncentrować energię, zrzuca model z jego niecki 0.92, a trening stabilizuje się ponownie dopiero przy $\alpha \approx 8\text{--}12$.



Rysunek 9: **Koncentracja energii przenosi się na CIFAR-10 tylko dla kierunku ślepego dla etykiety i jest opłacana dokładnością.** Po lewej: koncentracja energii (energy_frac względem poziomu losowego) w funkcji siły dopasowania α . Sink wysokoczęstotliwościowy (niebieski) rośnie od $1.5 \times$ przy $\alpha = 0$ do szczytu $44 \times$ przy $\alpha = 6$, utrzymując $23\text{--}28 \times$ przy $\alpha = 8\text{--}12$; kierunek losowy (czerwony) nigdy nie opuszcza linii losowej. Po prawej: te same punkty względem dokładności, odsłaniające kompromis i jego niemonotoniczność — dokładność spada przy $\alpha = 2$ (0.38), wraca do 0.69 blisko $\alpha = 12$, po czym załamuje się przy $\alpha = 32$ (0.35). Użyteczny obszar pracy to $\alpha \approx 8\text{--}12$: $\sim 23\text{--}28 \times$ poziomu losowego przy dokładności 0.67–0.69.

Mechanizm jest też stabilny względem budżetu ataku na CIFAR (pytanie Q4 z etapu 3): metryki koncentracji utrzymują się przy zmianie ε , więc efekt nie jest artefaktem jednego budżetu (Rysunek 10).

CIFAR-10 sink alignment vs attack budget: aligned fine-tune does not beat the converged baseline at healthy accuracy



Rysunek 10: **Stabilność metryk względem budżetu ataku ε .** support_cos, mass_frac i energy_frac w funkcji budżetu L_2 , dla bazy i sieci dopasowanej. Krzywe są gładkie i monotoniczne, bez efektów progowych: ten (niewielki) sygnał obecny na CIFAR jest spójny w różnych budżetach, a nie pojawia się tylko przy jednym starannie dobranym ε .

5. Synteza i werdykt

Wątki splatają się w trzyczęściową opowieść.

1. **Rozpoznawalnego sinka wizualnego nie da się narysować na CIFAR.** W pięciu mechanizmach i sześciu wzorcach, na w pełni zbieżnej sieci 0.92 przy szerokości do $2 \times$, żaden wzorec nie zostaje odtworzony (support_cos ≈ 0 , masa na poziomie losowym). To *scharakteryzowana* niemożliwość, z jawnie wykluczonymi czynnikami pojemności i wymiarowości.
2. **Koncentracja energii przenosi się na CIFAR** — ale tylko dla kierunku ślepego dla etykiety (wysokoczęstotliwościowego), na poziomie $23\text{--}28 \times$ losowego i nie za darmo: kosztuje dokładność ($0.92 \rightarrow \sim 0.68$) wzdłuż niemonotonicznej granicy. Kierunek losowy nie osiąga niczego.
3. **Środowisko toy dowodzi czystej granicy.** Gdy istnieją naprawdę nieużywane wymiary, koncentracja sięga $36\text{--}187 \times$ poziomu losowego przy niemal darmowej dokładności i jest odporna na zmianę budżetu — to wyidealizowana wersja wyniku (2), z tą samą granicą: znak nigdy nie jest kontrolowany.

Jedno zdanie tłumaczy całość: *gradient sieci względem wejścia może albo kodować klasę, albo wskazywać ustalony kierunek, ale nie jedno i drugie naraz*. Koncentracja klasy detekcyjnej na podprzestrzeni ślepej dla etykiety jest realna i sterowalna, za cenę dokładności; widoczny, znakowany rysunek jest blokowany właśnie przez to napięcie.

Uwaga o zakresie: specyfikacja wskazywała CIFAR-100 na etap 3. Świadomie pozostaliśmy przy CIFAR-10 i środowisku toy — skoro efekt nie daje rysunku wizualnego na łatwiejszym zbiorze, trudniejszy go nie uratuje, a środowisko toy izoluje mechanizm znacznie czyściej. Potwierdzenie na CIFAR-100 pozostaje odłożonym zadaniem niskiego ryzyka.

6. Wnioski i kierunki przyszłych prac

Postawiliśmy sobie za cel sprawienie, by model zdradził atakującego, zmuszając ataki gradientowe white-box do narysowania ustalonego symbolu wizualnego. Uczciwy wynik jest taki: silna wersja tego celu jest nieosiągalna z zasadniczych powodów, podczas gdy użyteczna wersja słabsza — zmuszenie ataku do wejścia w znaną, ślepą dla etykiety podprzestrzeń — jest osiągalna i została pokazana zarówno w środowisku toy, jak i na CIFAR-10. Wkładem jest ostra granica między tymi dwoma.

Naturalnie wynika z tego kilka konkretnych kierunków.

1. **Detektor zamiast rysunku.** Osiągalny efekt to dokładnie to, czego potrzebuje detektor: rzutować perturbację wejścia na znaną bazę i sygnalizować anomalną energię. Omija to całkowicie podwójną rolę gradientu i jest najbardziej obiecującym następnym krokiem.
2. **Rzeźbienie gradientu poza rozmaitością.** Dokładność ogranicza model jedynie na rozmaitości danych, podczas gdy atak porusza się głównie poza nią. Strata przypinająca funkcję na rozmaitości (składnik KL względem zamrożonego klasyfikatora), a nagnająca gradient tylko w odwiedzanych przez atak punktach poza rozmaitością, to pierwsze sformułowanie, w którym dopasowanie i dokładność nie muszą walczyć o ten sam gradient. Pytaniem otwartym jest koszt krzywizny.
3. **Trening wyznaczonego UAP i wąskie gardła architektury.** Trenować sink jako ustalony, kontrolowany co do tempa, uniwersalny kierunek zwiększający stratę [6] (łagodniejszy niż wymuszona etykieta), albo zbudować niskorangowe wąskie gardło wejścia, którego atakowalna podprzestrzeń zawiera sink z konstrukcji.
4. **Ujęcie przez cechy odporne.** Fakt, że tylko kierunek ślepy dla etykiety koncentruje energię, łączy się wprost ze spojrzeniem na przykłady adversarialne przez cechy odporne i nieodporne [7]; scharakteryzowanie, które podprzestrzenie są „darmowe” dla danego zbioru, pozwoliłoby z góry przewidzieć osiągalną koncentrację.

Bibliografia

- [1] Christian Szegedy *et al.*, „Intriguing Properties of Neural Networks”, w *International Conference on Learning Representations (ICLR)*, 2014.
- [2] Ian J. Goodfellow, Jonathon Shlens, i Christian Szegedy, „Explaining and Harnessing Adversarial Examples”, w *International Conference on Learning Representations (ICLR)*, 2015.
- [3] Aleksander Mądry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, i Adrian Vladu, „Towards Deep Learning Models Resistant to Adversarial Attacks”, w *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Jonas Rauber, Wieland Brendel, i Matthias Bethge, „Foolbox: A Python Toolbox to Benchmark the Robustness of Machine Learning Models”, 2017.
- [5] Tianyu Gu, Brendan Dolan-Gavitt, i Siddharth Garg, „BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain”, 2017.
- [6] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, i Pascal Frossard, „Universal Adversarial Perturbations”, w *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [7] Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, i Aleksander Mądry, „Adversarial Examples Are Not Bugs, They Are Features”, w *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] Alex Krizhevsky, „Learning Multiple Layers of Features from Tiny Images”, University of Toronto, 2009.